UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

FLUTTER



JULIO 2026-DICIEMBRE 2026

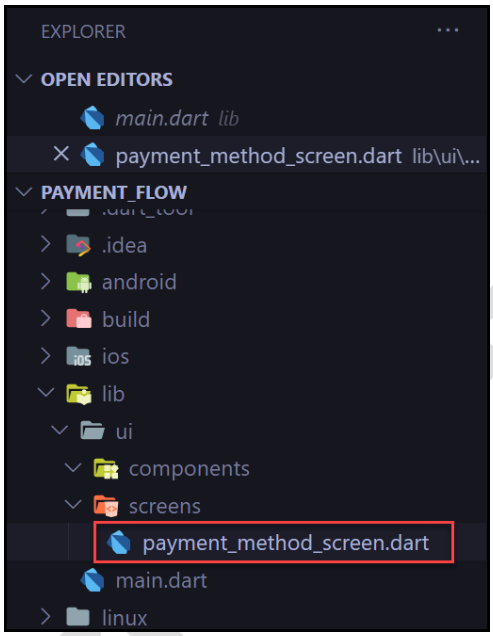
16/10/2026

Ejercicio # 1

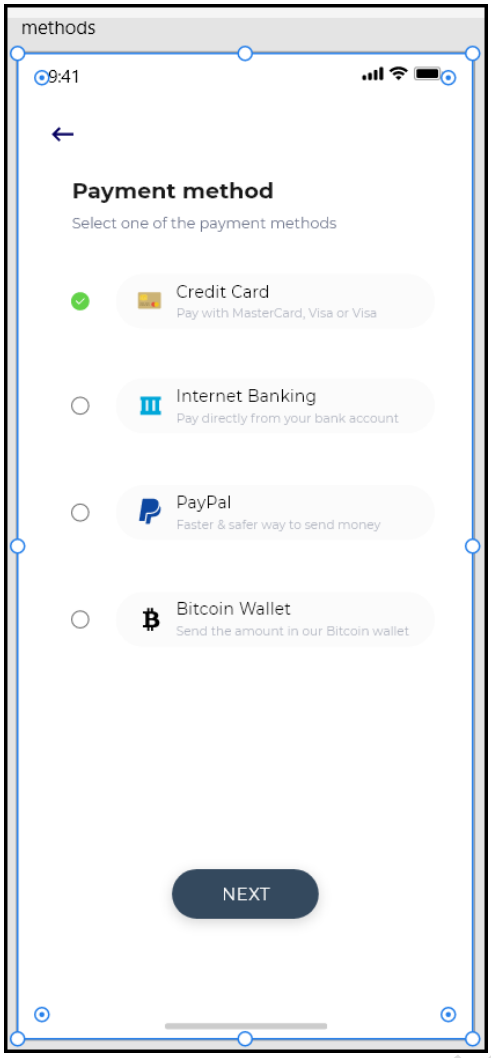
En esta práctica vamos a empezar el diseño de la primera pantalla, la pantalla de los métodos de pago, para la aplicación en Flutter.

Métodos de pago #1

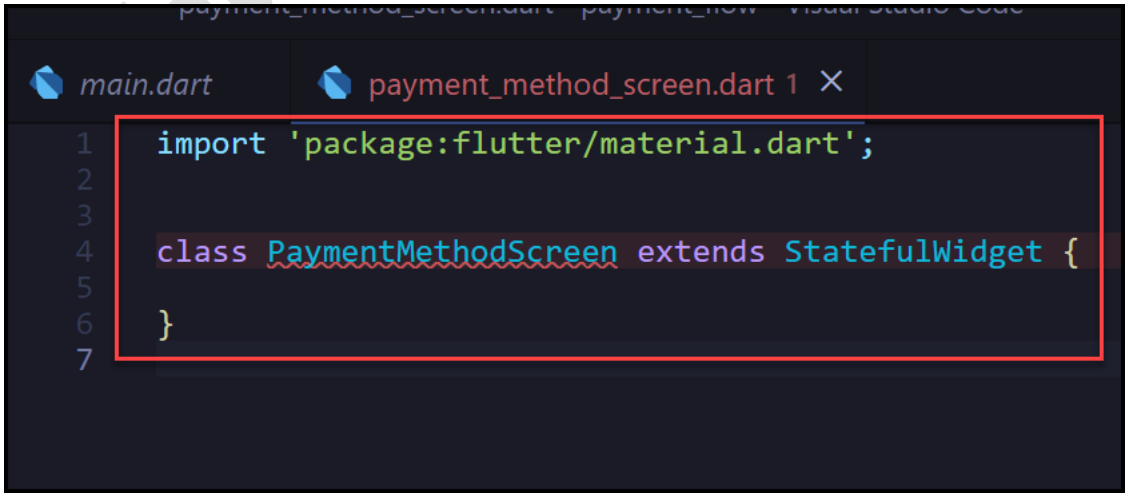
1. En esta práctica vamos a empezar con el diseño de la primera pantalla del proyecto, para esto, **dentro** de la carpeta **screens** **cree el archivo** con el nombre **payment_method_screen.dart** (método de pago) para la primera pantalla.



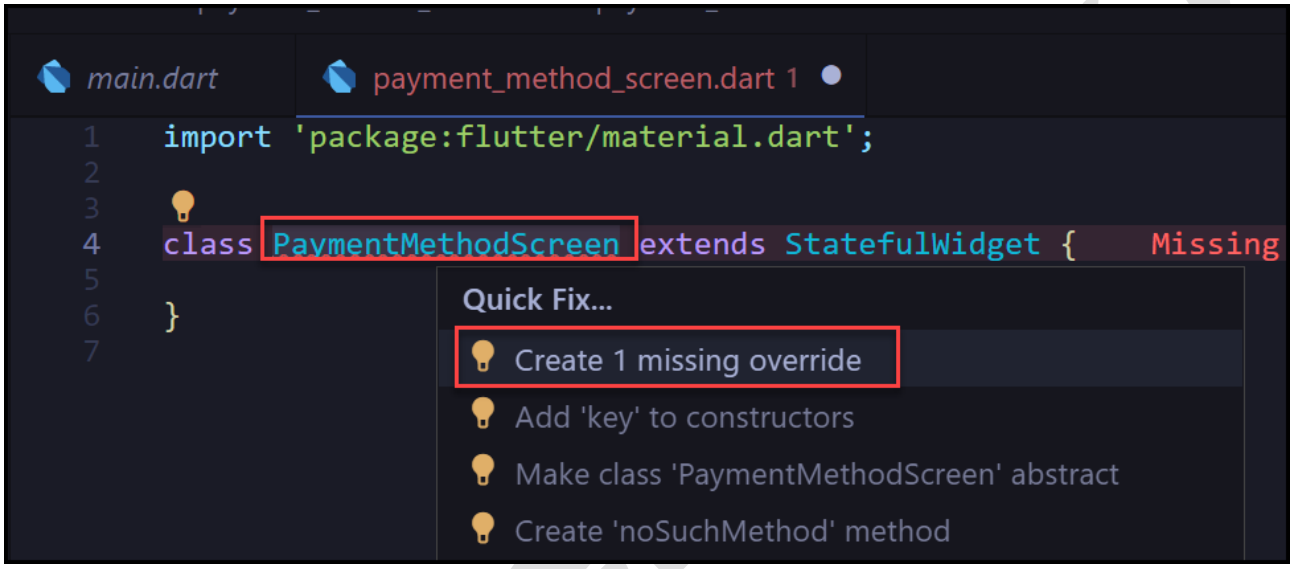
2. En el archivo **payment_method_screen** vamos a agregar la **primera pantalla** de la aplicación, según el **mockup** descargado sería la pantalla que permite seleccionar el **método de pago**, es decir vamos a crear la siguiente pantalla:



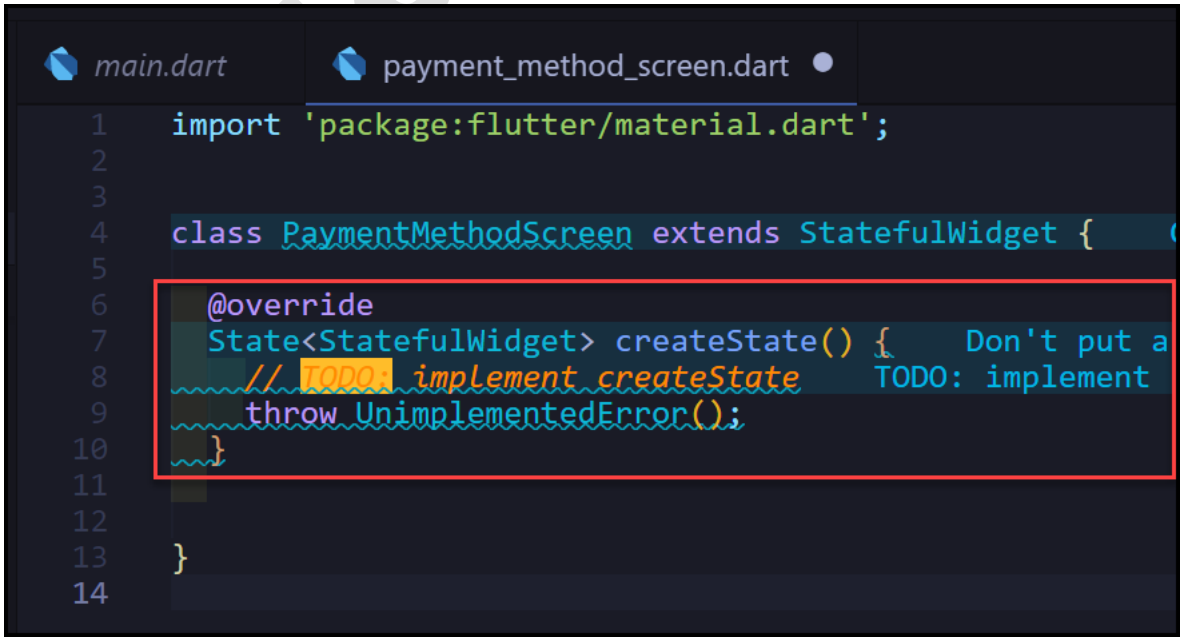
3. Para esto, en el archivo creado, en la primera línea **importe** el paquete **material.dart** y además cree el widget (clase) y herédela de la clase (widget) de tipo **StatefulWidget** llamado: **PaymentMethodScreen**. Esto se muestra a continuación.



4. Nos ayudamos del IDE de **Visual Studio Code** para **generar** el método **createState()** que debe tener en **forma obligatoria** una clase que se extienda de la clase **StatefulWidget**. Para esto ubique el **mouse** sobre la clase **PaymentMethodScreen** y **presione** las teclas **CONTROL + PUNTO** (CONTROL + .), en la **ventana emergente** que aparece **seleccione** la opción: **Create 1 missing override**.

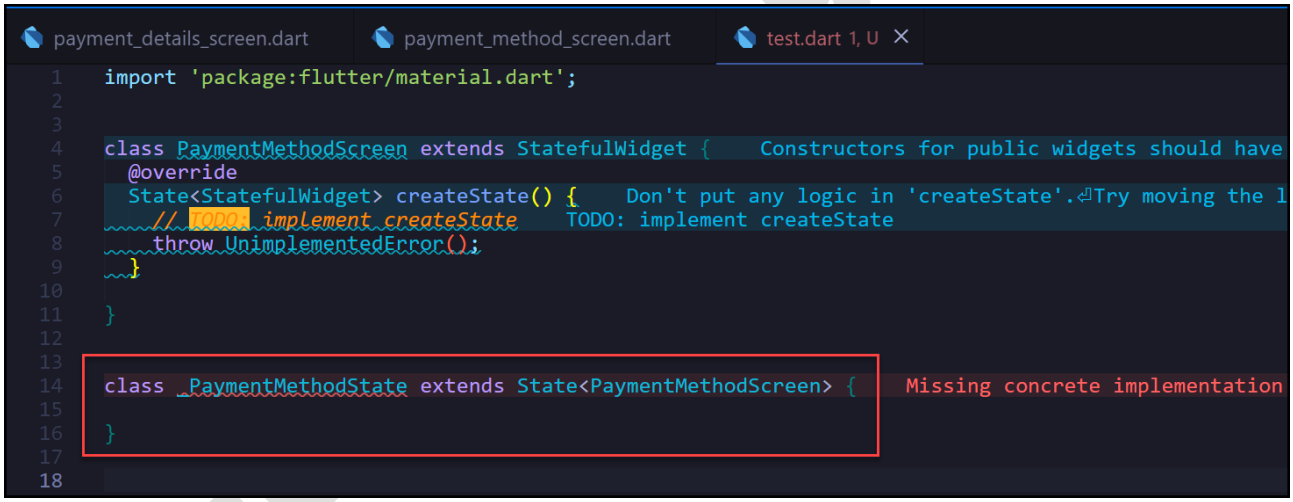


5. Esto genera el método **createState()** para administrar el **estado** del **Widget** (clase) **PaymentMethodScreen**.



Nota	• Se sugiere que la clase que hereda de la clase de tipo **StatefulWiget** termine con el sufijo **Screen** o el sufijo **Page**, esto permite identificar la clase fácilmente y saber que es una clase que representa una **pantalla.**

6. En el mismo archivo, al final, **cree** la clase **_PaymentMethodState** y herédela de la clase **State** y entre los **símbolos mayor y menor que** (<>) indique cual es la clase para la cual se desea **administrar el estado** con esta clase. La clase que va entre estos símbolos de mayor y menor que (<>) debe ser de tipo **StatefulWidget**, para el ejemplo la clase se llama **PaymentMethodScreen**. Tome en cuenta el guion bajo (_) en el nombre de la clase, en Dart esto significa que la clase es de tipo privada (private).

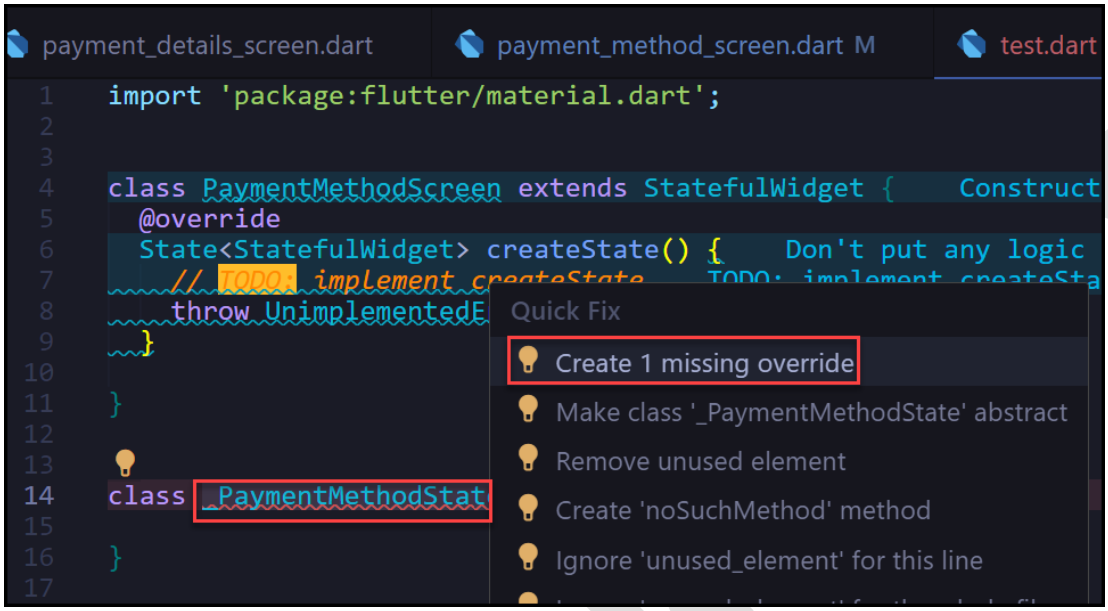


Nota	• Se sugiere que la clase que administra el estado de la clase de tipo **StatefulWiget** sea de tipo privado (con un guion bajo en Dart) y termine con el sufijo **State** para identificarla de otras clases fácilmente.

7. Para el siguiente paso nos ayudamos nuevamente del IDE de **Visual Studio Code** para **generar** el método **build()** para la clase **_PaymentMethodState**. Para esto

ubique el mouse sobre la palabra **_PaymentMethodState** y luego **presione** las teclas **CONTROL + PUNTO** (CONTROL + .), en la **ventana emergente** que aparece **seleccione** la opción: **Create 1 missing override**. Esto genera el siguiente **código**.

De la ventana emergente seleccione la opción **Create 1 missing override**.

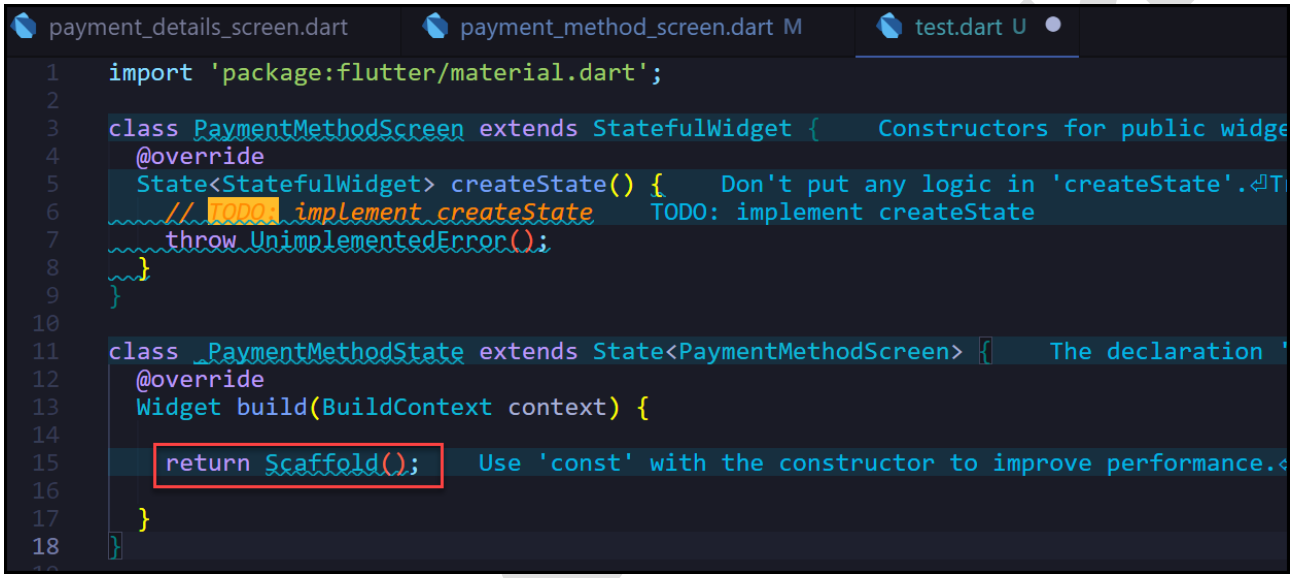


Se genera el método **build**.



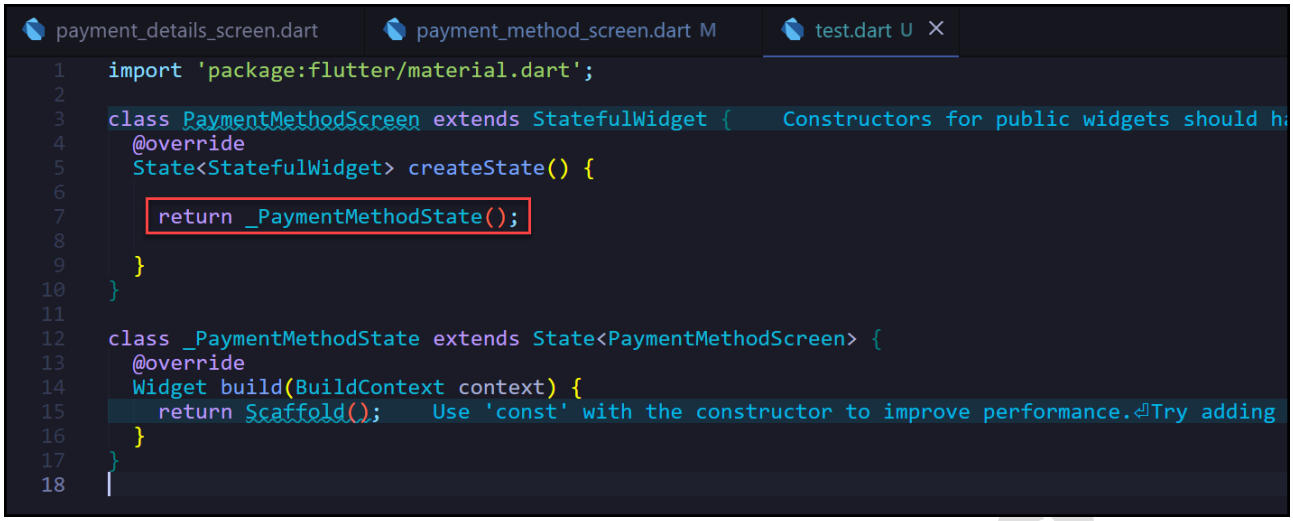
Nota	• El método **build** es donde se genera el código para crear la **interfaz de usuario** de una pantalla o formulario en Flutter.

8. En el método **build** creado, **borre** el código generado por el IDE y por el momento **retorne** un **widget** de tipo **Scaffold**. El método build es donde genera toda la UI.



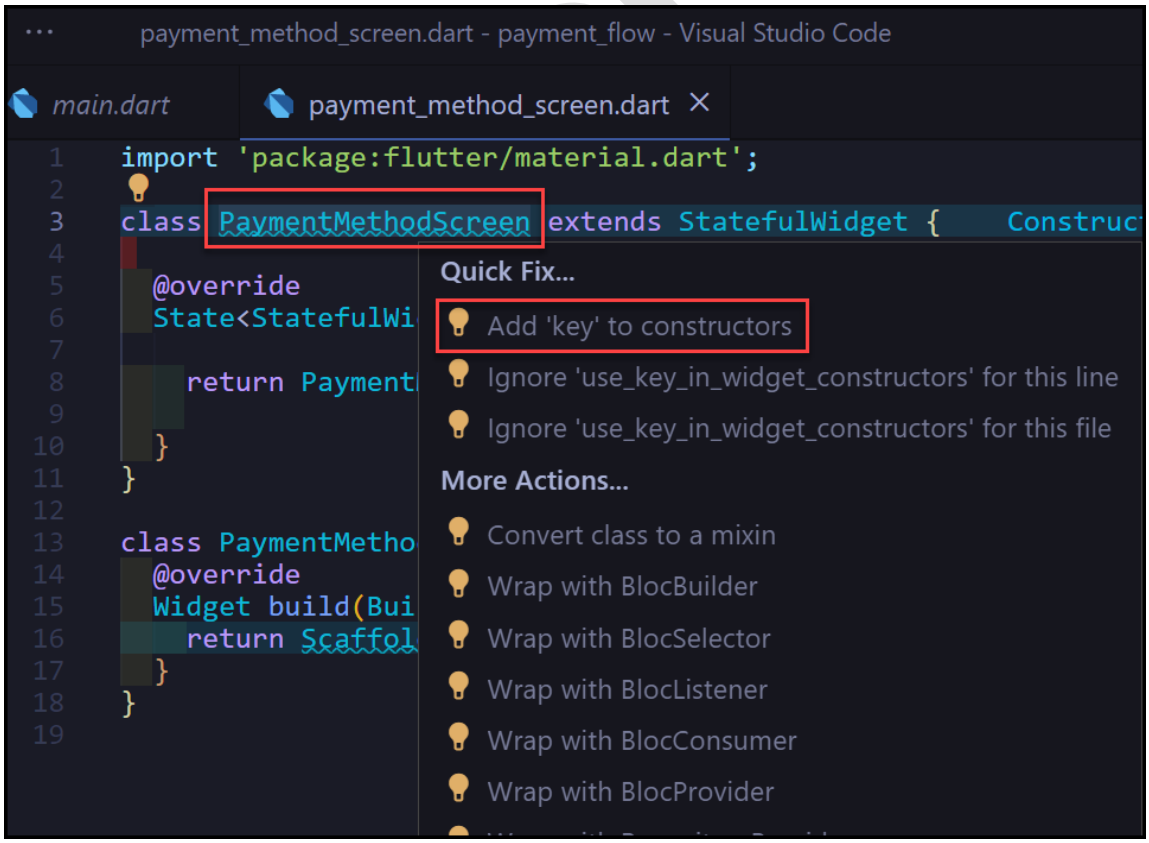
Nota	• El **Scaffold** es un widget proporcionado por el paquete material de Flutter (**material.dart**), que implementa la **infraestructura básica** de **Material Design** para la UI de la aplicación. Permite a los desarrolladores establecer una estructura de página **uniforme** y **cohesiva** en la aplicación. El uso del **Scaffold** es casi imprescindible en las apps que siguen el diseño Material (Material Design).

9. **Borre** el código **generado** por el IDE de Visual Studio Code del método **createState** de la clase **PaymentMethodScreen** y **retorne** una **instancia de la clase** que va a **administrar el estado** de la clase **PaymentMethodScreen**, esto es, una instancia de la clase **_PaymentMethodState**.

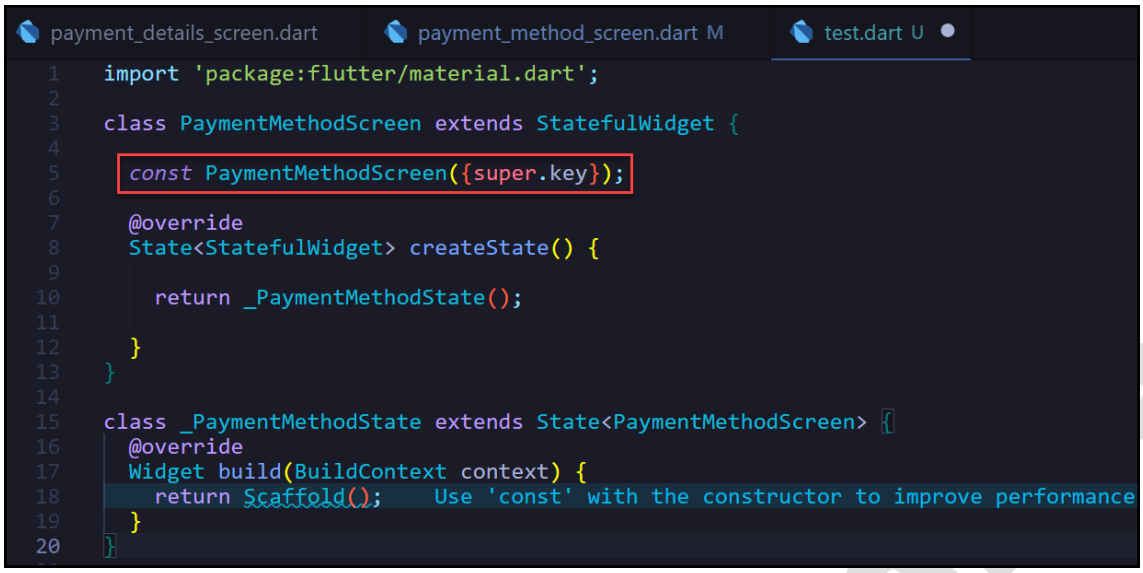


10. Para quitar los **warnings** presentes en el código anterior, genere el **constructor (key)** de la clase **PaymentMethodScreen**, para esto ponga el mouse sobre la clase **PaymentMethodScreen**, **utilice** el IDE de **VS Code** (presione CONTROL + PUNTO). En la ventana emergente **seleccione** la opción: **Add key to constructors**.

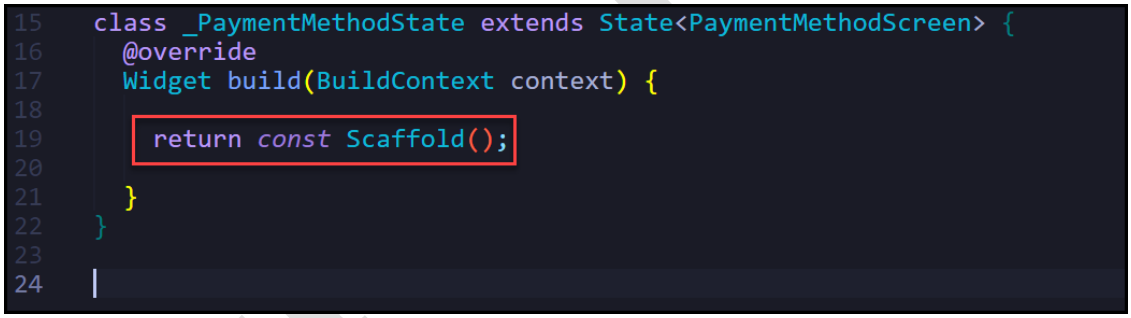
Selección la opción indicada.



Constructor con el **parámetro key** generado.



11. Por el momento **agregue** la palabra **const** a la instancia de la clase **Scaffold** del método **build** de la clase **_PaymentMethodState** para **eliminar** el **warning**. Aunque en las siguientes prácticas la palabra **const** se volverá a quitar según sea necesario.



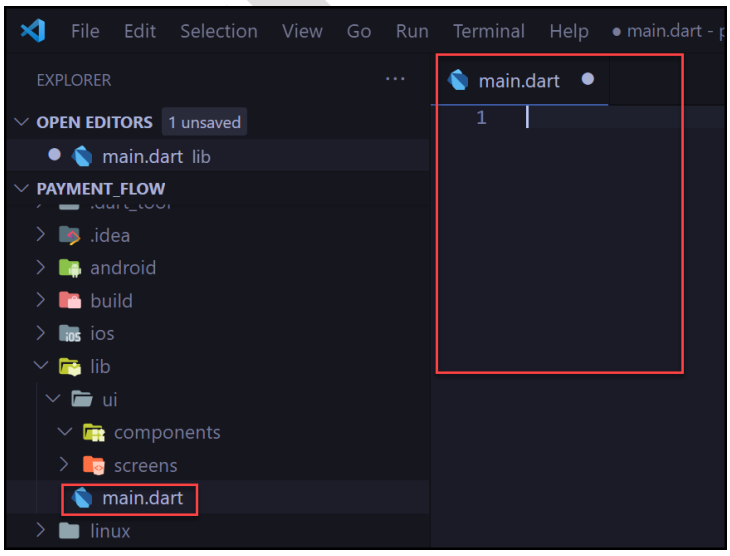
Nota	• Actualmente me sugiere el IDE que coloque elementos con **const.** Los **Warnings** son la forma que el IDE tiene de hacer más "correcto" el código, pero muchas veces no va de acuerdo con nuestra forma de programar o con la versión terminada del programa en sí; por ejemplo, le pedirá poner el Texto como **constante,** pero el texto no siempre será el mismo, depende de los datos que reciba, entonces en ese caso no conviene ponerlo como constante. Una opción es que ignore los **warnings** por lo menos hasta que tenga terminado el

	programa. • La otra alternativa es ir **agregando** y **quitando** la palabra reservada **const** según vaya apareciendo y creando el código. En este caso vamos a elegir esta opción para el proyecto.

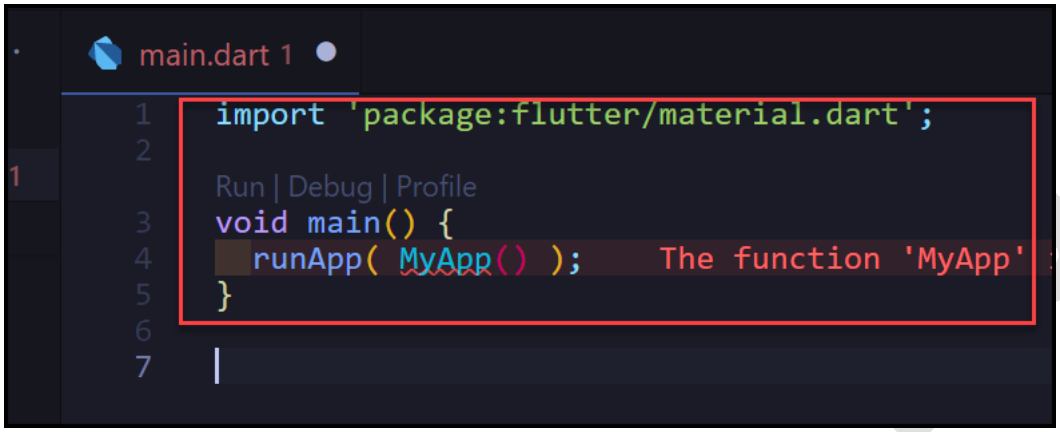
12. El archivo **payment_method_screen.dart** sin **warnings** una vez eliminada la palabra reservada **const** tendría el siguiente código.



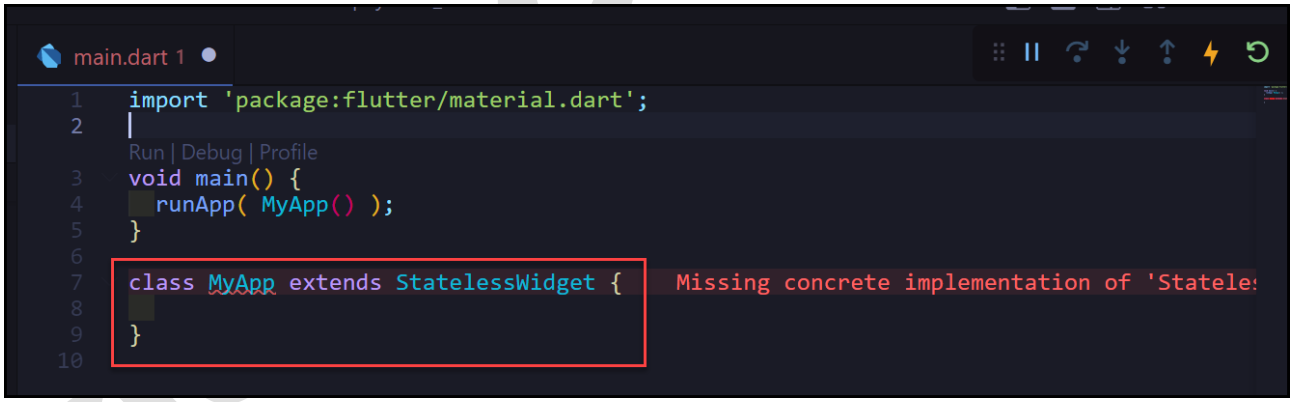
13. Ahora **vaya** al archivo **main.dart** y **elimine** todo el código generado por el asistente.



14. En este archivo (**main.dart**) **digite** el siguiente código para utilizar el paquete **material** (que es donde se encuentran la mayor parte de los controles para crear la UI para una app Android que utiliza **Material Desing**) y llame al método **main** que es el punto de inicio de una app en Dart.



15. Ahora a continuación en el mismo archivo, **cree** una clase con el nombre **MyApp** (puede ser cualquier nombre) del tipo **StatelessWidget** donde vamos a tener la **lógica principal** de la clase (Widget) llamada **MaterialApp** que se creará máss adelante.

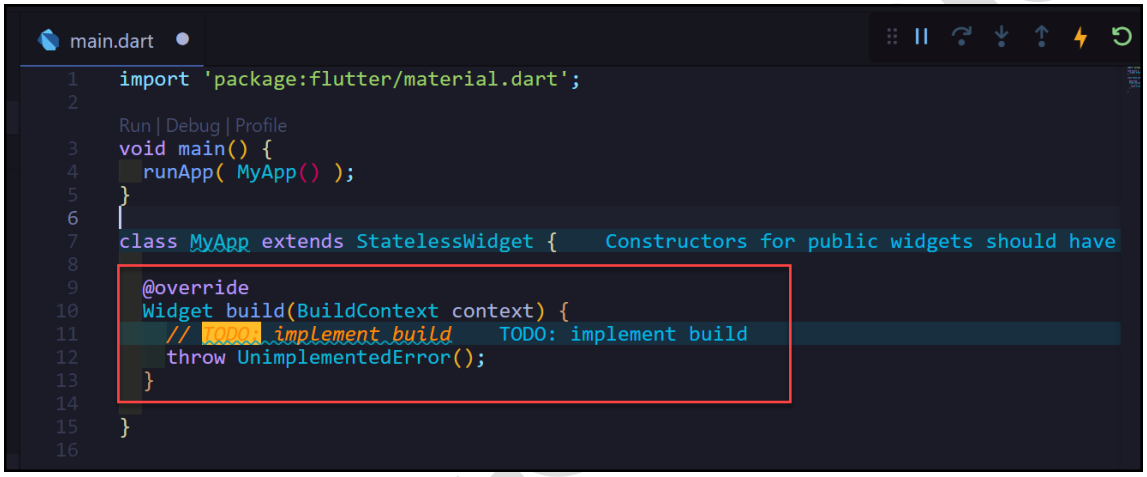


16. **Dentro** de esta **clase** ponga el **cursor** en la clase **MyApp** y haga un **override** (**sobrescriba)** el método **build**, para esto nos ayudamos del **Visual Studio Code** (presione CONTROL + PUNTO) y de la ventana emergente seleccione **Create missing override**.

Seleccione la opción indicada.



Se genera la estructura del método **build**, donde se debe crear la interfaz del widget.



17. Como se utiliza **Material Desing** para el diseño la UI, el método **build** debe **retornar** una instancia de la clase **MaterialApp**, por lo tanto, **modifique** el código de este **método**, de la siguiente forma.



Nota	• **MaterialApp** es un widget que nos proporciona un **layout** orientado a **Material Design.** • El widget **MaterialApp** tiene como atributos entre otros: **title** el título de la app, **theme:** el tema que vamos a usar, **home:** en el atributo home tenemos que pasar la primera 'pantalla' que queremos mostrar.

18. **Configure** algunos atributos del widget **MaterialApp**, **primero** establecemos un **título** (title) para el widget, por ahora también **ponemos** la palabra **const** donde sea necesario, esto luego se ira **eliminando** según el IDE vaya solicitando. El código digitado se visualiza en la siguiente figura.

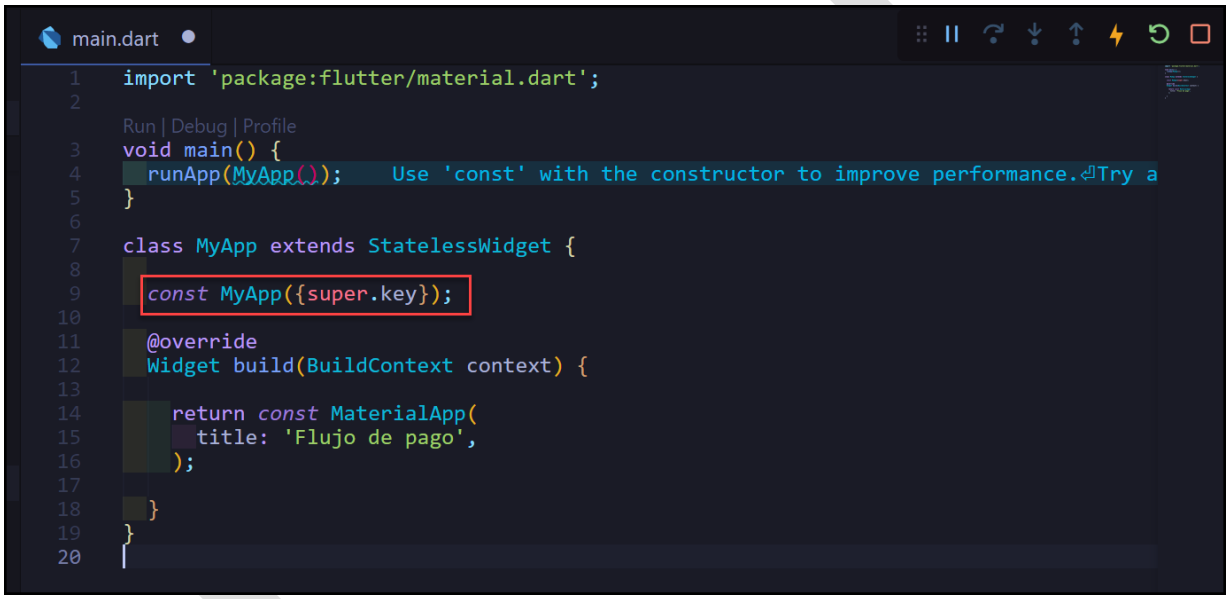


19. Para quitar los **warnings** genere el **constructor (key)** de la clase **MyApp**, para esto ponga el mouse sobre la clase **MyApp**, **utilice** el IDE **Visual Studio Code** para automatizar el proceso (presione CONTROL + PUNTO), de la ventana emergente seleccione la opción **Add 'key' to constructors**.

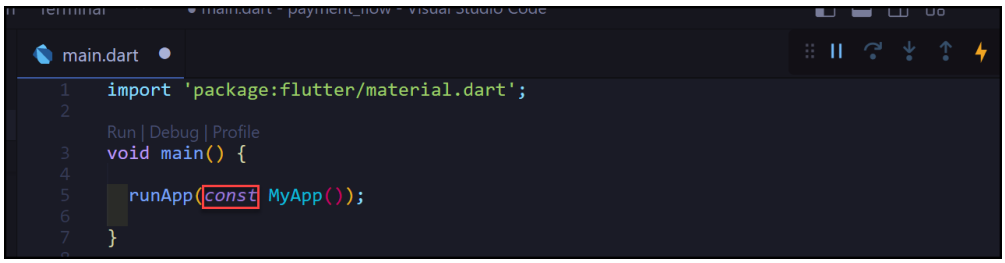
Seleccione la opción indicada.



Se crear el constructor y se agrega el parámetro (key).



20. Para quitar los warnings **agregue** la palabra reservada **const** en la **instancia** de la clase **MyApp** puesto que ahora la clase tiene un constructor constante, esto en el método **runApp**.



Nota	• Lo primero que nos encontramos en el fichero **main.dart**, es la función **main()** que es la encargada de ejecutar nuestra aplicación con el método **runApp()**. • El método **runApp()** y un parámetro ejecuta la pantalla inicial de una aplicación, la pantalla es la instancia que se pasa como parámetro al método.

21. El **código** del archivo **main.dart** sin warnigns se muestra a continuación.

main.dart

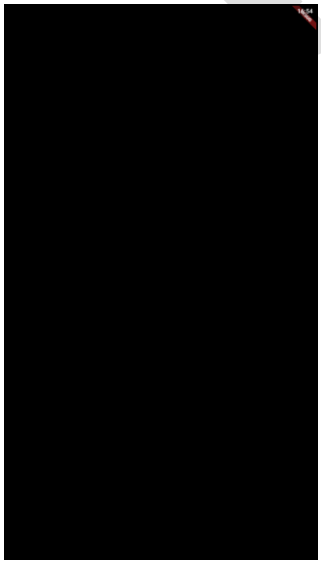
```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Flujo de pago',
    );
  }
}
```

22. **Ejecute** el proyecto. Este debería tener la siguiente apariencia.



23. Ahora en el mismo archivo **main.dart**, en la clase **MyApp**, vamos a **establecer las rutas** (menú de navegación del proyecto). Las rutas son **instancias de tipo mapa** (Map) que contienen datos del tipo: **par->valor**. Para las rutas vamos a poner una **clave** (key) y el **nombre de la instancia** (ventana) que deseamos mostrar cuando se presione en botón de la barra de navegación, en este caso será una **instancia** de la clase **PaymentMethodScreen** la que se desea mostrar. Esto lo hacemos en el método **build**, luego del parámetro con nombre **title**, de la clase **MaterialApp**. A continuación, se muestra el código necesario para esto.

main.dart M

```dart
You, 6 seconds ago | 1 author (You)
import 'package:flutter/material.dart';      You, 15 hours ago • Terminado practica 002

import 'package:payment_flow/ui/screens/payment_method_screen.dart';

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}

You, 6 seconds ago | 1 author (You)
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {

    return MaterialApp(

      title: 'Flujo de pago',

      routes: {
        '/': (context) {
          return const PaymentMethodScreen();
        }
      },

    ); // MaterialApp
  }
}
```

Nota	• Note que en la **segunda línea** del código anterior se ha **importado** el archivo donde está definido la clase **PaymentMethodScreen**. • Para que no se genere **warnings** en el código anterior, **por el momento** en la sentencia **return** de **routes** se usa la palabra reservada **const** para regresar el **constructor constante** de la clase **PaymentMethodScreen**. Se usa **const** por el momento debido a que la clase **PaymentMethodScreen** no cambia durante toda la aplicación (es

	decir es **inmutable)**, esto es así por el momento, pero esto **puede cambiar a medida que necesitemos** que algo cambie en la pantalla **PaymentMethodScreen**, esto hará que debamos eliminar la palabra reservada **const** de esta línea de código. • Luego de esto la clase **MaterialApp** **debe generar un error** por la palabra reservada **const**, por lo tanto, esta **no puede ser constante** ya que la ruta se establece dinámicamente, **es decir cambia en tiempo de ejecución**. Por lo que **se ha eliminado** la palabra reservada **const** de **MaterialApp**, esto se puede visualizar en el código anterior. Revise el código anterior de la clase para mayor referencia.

24. El **código** anterior que está dentro del parámetro por nombre **routes** se lo puede también **escribir** usando **expresiones lamba**. Esto sería de la siguiente forma.

main.dart

```dart
import 'package:flutter/material.dart';
import 'package:payment_flow/ui/screens/payment_method_screen.dart';

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {

    return MaterialApp(
      title: 'Flujo de pago',

      routes: {

        '/': (context) => const PaymentMethodScreen(),

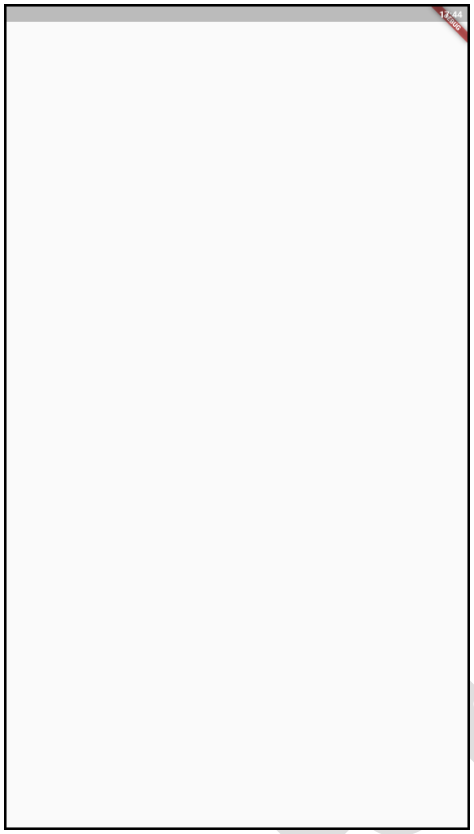
      },

    ); // MaterialApp
  }
}
```

Nota	• Puesto que la **segunda forma de escribir el código** es la más **habitual** vamos a utilizar esta forma, con **expresiones lambda**.

25. El **código** dentro del parámetro **routes**, establece que al arrancar la aplicación la **pantalla** (ventana) **por defecto**, primera pantalla (esto se expresa con el símbolo: **'/'**) o **ventana de inicio** (parte izquierda), es una instancia de la clase **PaymentMethodScreen** (parte derecha). Si **se ejecuta** la aplicación, se puede verificar cual es la **pantalla** que se muestra en primer lugar al arrancar la aplicación.

Pantalla de inicio.



26. **Para garantizar** que una instancia de la clase (ventana, widget) **PaymentMethodScreen** sea la **pantalla por defecto o de inicio**, se debe usar el **parámetro initialRoute** con el valor **'/'**, en la clase (Widget) **MaterialApp**, de la siguiente forma.

Flutter-Clay Aldás

main.dart

```dart
import 'package:flutter/material.dart';
import 'package:payment_flow/ui/screens/payment_method_screen.dart';

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

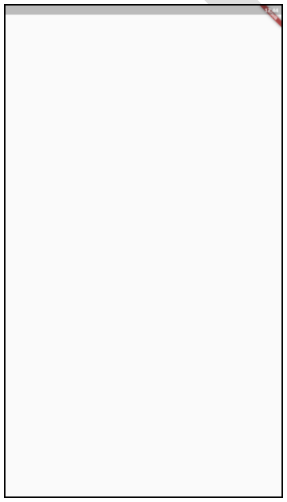
  @override
  Widget build(BuildContext context) {
    return MaterialApp(

      initialRoute: '/',

      title: 'Flujo de pago',
      routes: {
        '/': (context) => const PaymentMethodScreen(),
      },
    ); // MaterialApp
  }
}
```

Nota	• Una forma de entender cómo se utiliza las **rutas en Flutter**, es comparar como se utilizan las rutas en las **páginas Web**, es decir se tiene una **dirección** (url) y **apuntamos** específicamente hacia una **instancia** de la clase (ventana, página) **PaymentMethodScreen** que deseamos mostrar la presionar en la dirección (url), la cual se encuentra en el archivo **payment_method_screen.dart**.

27. **Ejecute** y pruebe la aplicación.



Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

Visual Studio Code para el desarrollo de flutter
https://devjaime.medium.com/visual-studio-code-para-el-desarrollo-de-flutter-256dbd13b453
Flutter I: Instalación e Introducción
https://medium.com/@XensS/flutter-i-instalaci%C3%B3n-e-introducci%C3%B3n-878c155ab4a8#.1k7aqghea
Stateful vs Stateless en Arquitectura
https://refactorizando.com/stateful-vs-stateless-arquitectura/
Árbol de Widget y Contexto en Flutter
https://es.linkedin.com/pulse/%C3%A1rbol-de-widget-y-contexto-en-flutter-oliver-yunior-mendez
MaterialApp class
https://api.flutter.dev/flutter/material/MaterialApp-class.html
Scaffold class
https://api.flutter.dev/flutter/material/Scaffold-class.html
Descubriendo el Widget Scaffold de Flutter
https://www.ricardogottheil.com/descubriendo-el-widget-scaffold-de-flutter/
Poniendo los métodos build a dieta: trucos y consejos para un código de UI más limpio en Flutter
https://medium.com/comunidad-flutter/poniendo-los-m%C3%A9todos-build-a-dieta-trucos-y-consejos-para-un-c%C3%B3digo-de-ui-mas-limpio-en-flutter-cbc1a158723f
Building Design Systems in Flutter
https://betterprogramming.pub/building-design-systems-in-flutter-d52d66004070
Clean Your UI Code in Flutter
https://medium.com/@ximya/clean-your-ui-code-in-flutter-7c58bf3e267d
Haz que tu app de Flutter pase de aburrida a atractiva
https://codelabs.developers.google.com/codelabs/flutter-boring-to-beautiful?hl=es-419#0
Construyendo sistemas de diseño en Flutter
https://www.desarrollolibre.net/blog/flutter/construyendo-sistemas-de-diseno-en-flutter

flutterClay Aldás